

Data Pipeline Roadmap

How we get from 541 Word opinions to a frozen 20-case pilot, then Postgres + pgvector — without rebuilding the app.

Court Opinions Analyzer, Phase II · Improving Reliability in Legal AI Through Multi-Agent Verification
Prepared by Enock Katenda (Infrastructure) · For Alex Ji, Leandro Ardiles, Tafadzwa Nyemba
30 September 2026

1. Why this document exists

Capstone II is an experiment: keep the Capstone I application as **System A**, add a multi-agent layer as **System B**, and measure whether agent disagreement catches bad legal answers better than self-reported confidence.

The real corpus is not currently in the app. It lives as **541 LEXIS Word files**. The app today only uploads Parquet into SQLite. The later Evidence Checker will need **cited spans**, which means chunked text with offsets — and eventually pgvector.

Decision: we do not start a new repository or a new product. The pipeline is added inside `us-court-decisions-analyzer`. It feeds the existing upload / module-launch / validator workflow. Postgres and pgvector come after a usable 20-case set exists, not before.

2. Research constraint (do not skip this)

Every pipeline choice has to serve the team plan already agreed in the 23 September status report:

Phase	Capstone II focus	Timing	Pipeline must deliver
1–2	Reproduce baseline; benchmark GPT-OSS 20B / 120B / Qwen 3.8 27B	Weeks 1–2 24 Sep – 7 Oct	Frozen 20-case Parquet the current app can upload
3–4	Build two readers + adjudicator; compare System A vs B	Weeks 3–5	Postgres + clean case rows; retrieval stub if needed
5	Temporal agent + evidence checker	Week 6+	Chunks, character offsets, pgvector search

Q1 (type of election) is the control — Capstone I was strong here. Q2 (days between trial ruling and election) is the known failure mode. Ground truth is Tafadzwa’s; models and agents are Alex’s; the live app and API account are Leandro’s; data plumbing is Enock’s.

3. How the stages run

Stage 0 Word → 20-case Parquet	Stage 1 Clean metadata + splits	Stage 2 SQLite → Postgres	Stage 3–4 Chunks + pgvector + retrieve
-----------------------------------	------------------------------------	------------------------------	---

Stage 0 — Frozen pilot (this week) DONE

Goal. Unblock the 7 October benchmark without waiting for Postgres.

How it runs. A CLI walks `current_docs/MS Word files (544 files total)/`, extracts text, hashes files, groups duplicates, and writes a Parquet file whose columns already match the existing upload parser.

Command (from `backend/`): `python -m app.ingest.cli --pilot 20`

Input	Output	Who uses it
541 LEXIS .docx files	<code>backend/data/pilot_20.parquet</code>	Leandro — upload in the existing admin UI
	<code>backend/data/pilot_20_manifest.json</code>	Whole team — frozen case list; do not silently replace
	<code>backend/data/inventory.csv</code>	Tafadzwa — start Q1/Q2 answer key

Result of the first run (30 Sep 2026). 541 files parsed, 0 failures, 540 unique opinions, 77 same-name alias groups (e.g. *Veasey v. Perry (2)*). Twenty unique cases selected with a length cap so they fit in an LLM context window (about 2k–48k characters of opinion text). All 20 have a parsed date; 17 have a docket number.

Still required from the team: Tafadzwa reviews the 20 names for Q1/Q2 suitability and writes the answer key. After that review, the manifest is frozen. If a case is a bad fit, we swap it in a new manifest version — we do not silently regenerate the list.

Stage 1 — Word → structured cases NEXT

Goal. Reliable metadata and opinion / dissent / concurrence splits for the whole corpus, still as a CLI. No new UI. No embeddings.

How it runs.

1. Extract paragraphs with `python-docx` / XML (already working).
2. Strip remaining LEXIS artifacts (reporter stars, counsel blocks, “End of Document”).
3. Parse caption fields with rules first: case name, court, date, citation, docket, judges. Use an LLM only if rules fail — do not spend Groq budget on 541 files.
4. Split body into opinion / dissent / concurrence.
5. Dedup by text hash; keep filename aliases as sources, not extra cases.
6. Write one canonical row per unique opinion. Leave `election_type` empty. Humans and agents answer Q1; ingest must not invent it.

Success. Unique opinions load (~450 after alias collapse). A parse-failure report exists. Docket / court / date quality is good enough for the experiment log.

When. 1–8 October, in parallel with the model benchmark.

Stage 2 — Move storage to Postgres WEEKS 3–4

Goal. Same 13-table workflow, now on Postgres, so pgvector can be added later. SQLite remains a Dummy-AI fallback.

How it runs.

1. Docker Compose: Postgres 16 + `CREATE EXTENSION vector;`
2. Point `DATABASE_URL` at Postgres. Add Alembic (already planned in the schema changelog).
3. Port existing tables unchanged. Do not redesign `court_cases` or `launch_module()`.
4. Add only ingest columns: `source_documents` (path, sha256, parse status) and on `court_cases`: source id, citation, raw text, character length.

Success. Login, project, module, validator, and feedback flows still work. Official experiments can load the frozen 20 from Postgres instead of a one-off Parquet upload.

Stage 3 — Chunking + pgvector AFTER STAGE 1 IS CLEAN

Goal. Store opinion pieces with stable character offsets so an agent can cite a span and an Evidence Checker can verify it.

How it runs.

1. Split on legal sections first (Opinion, Dissent, I., II.), then paragraphs, then 400–800 token windows with overlap.
2. Save `char_start` / `char_end` against the cleaned source text. Round-trip test: `raw_text[start:end] == chunk.text`.
3. Embed with a local/open model (e.g. nomic-embed-text). Do not spend the Groq experiment budget on embeddings.
4. Write `opinion_chunks` (vector column) and `ingest_runs` (files seen, cases written, errors).

What this is not. Not a chatbot. Not a replacement for sending the full opinion on the 20-case pilot. Readers still get the whole opinion for Q1/Q2. Retrieval is for long documents, span citation, and Phase 5 evidence checking.

When. Once Stage 1 metadata is trusted; target late October so it is ready before the Evidence Checker.

Stage 4 — Plug in without rewriting the app WITH SYSTEM B

Engineering rule from the 24 September slides: do not rewrite the frontend first. Replace only the single analysis call behind `launch_module()`.

Today	After the pipeline
Admin uploads one Parquet file	Admin can still upload Parquet (System A). Optional: “Load Codrington corpus” from ingest.
<code>CourtCase.opinion_text</code> is the whole document	Same column stays. Chunks sit beside it.
<code>_run_groq_ai_analysis()</code> sees full text	Unchanged for the 20-case benchmark. Later, an internal <code>POST /internal/retrieve</code> returns top-k spans.

4. Calendar — who does what while the pipeline runs

When	Enock (infrastructure)	Rest of team
30 Sep – 7 Oct Week 2 checkpoint	Stage 0 complete. Start Stage 1. Share this 20-case list.	Everyone runs Dummy AI baseline. Leandro: model IDs + API key. Tafadzwa: ground truth on the 20. Alex: agent interface spec.
8 – 21 Oct Weeks 3–4	Finish Stage 1. Stage 2 Postgres. Retrieval stub can return full-text chunks without embeddings.	Alex builds two readers + adjudicator behind <code>launch_module()</code> . Dummy mode first.
22 – 28 Oct Evaluate	Stage 3 embeddings on the frozen 20, then the rest of the corpus.	System A vs System B on the identical 20 cases. Same metrics: accuracy, disagreement, calibration, tokens, cost.
Week 6+	Retrieval quality, offset integrity, ingest-run metrics.	Temporal reasoning agent and Evidence Checker consume cited spans.

If Stage 3 slips, the experiment still runs. System A vs B only needs `opinion_text` on 20 cases. Vectors are an accelerator for evidence, not a gate on the research question.

5. Frozen pilot (proposed — pending Tafadzwa review)

Selected for election-related language, unique case identity, court diversity, and LLM-safe length. Not yet human-validated for Q1/Q2.

#	Case	Court
1	Republican Nat’l Comm. v. Wetzel	5th Cir.
2	Bognet v. Boockvar	W.D. Pa.
3	Minn. Voters Alliance v. Lake Cnty.	Minn. Dist. Ct.
4	Am. Council of the Blind of Ind. v. Ind. Election Comm’n	S.D. Ind.
5	League of Women Voters of S.C. v. Andino	D.S.C.

#	Case	Court
6	Shabazz v. N.Y.C. Bd. of Elections	N.Y. Sup. Ct.
7	Alliance for Retired Ams. v. Sec’y of State	Me. Sup. Jud. Ct.
8	Adair County Bd. of Elections v. Arnold	Ky. Ct. App.
9	State ex rel. Skaggs v. Brunner	S.D. Ohio
10	Tex. League of United Latin Am. Citizens v. Hughs	5th Cir.
11	League of Women Voters v. LaRose	S.D. Ohio
12	Republican Nat’l Comm. v. Schmidt	Pa. Sup. Ct.
13	Carson v. Simon	8th Cir.
14	New Ga. Project v. Raffensperger	11th Cir.
15	Self Advocacy Sol. N.D. v. Jaeger	D.N.D.
16	Moore v. Circosta	E.D.N.C.
17	Common Cause/Georgia v. Billups	11th Cir.
18	Paher v. Cegavske	D. Nev.
19	Carey v. Wis. Elections Comm’n	W.D. Wis.
20	Short v. Brown	E.D. Cal.

6. What we will not do

- Do not create a second GitHub project or a chatbot UI.
- Do not embed before metadata parse and dedup are trusted. Garbage vectors are expensive to redo.
- Do not send 541 full opinions through Groq to “extract everything.” Rules for captions; Groq budget is for the 20-case benchmark.
- Do not invent a new case schema that breaks `launch_module()`. Add columns; do not fork `CourtCase`.
- Do not treat filename as identity. Use content hash + parsed caption. Copies like `(2).docx` are aliases.

7. Ask of the team at the next meeting

1. **Tafadzwa:** review the 20-case list; confirm or swap; begin Q1/Q2 ground truth.
2. **Leandro:** upload `pilot_20.parquet` into a Capstone II project so everyone is on the same corpus.
3. **Alex:** keep agent interfaces independent of storage. Reader output still needs *cited evidence spans*; Stage 3 will make those spans real.
4. **Enock:** proceed to Stage 1 once the 20 are accepted.

Source documents: *Status Report and Plan — Capstone II* (23 Sep 2026) and *Capstone II — 24 September* slides. Code: `backend/app/ingest/` in the existing Court Opinions Analyzer repository. This note describes the infrastructure track only; it

does not change the System A vs System B evaluation design.